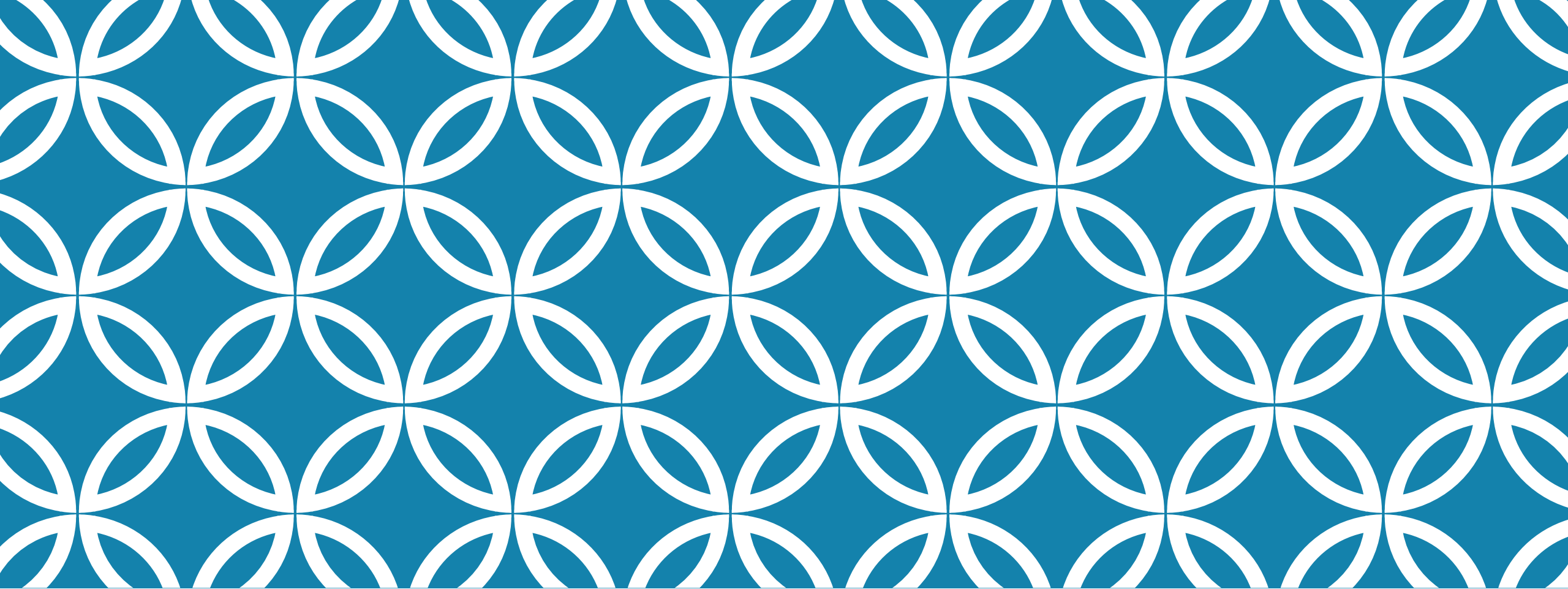




STATIC PROGRAM ANALYSIS

Lecture 16 – Call Graphs

CALL GRAPHS COMPLICATED

Static methods

- Resolve at compile-time
- One callsite matches one method definition
- E.g., `Math.power(x,y)` connects actual parameters of `x,y` to the formal parameters of method `power` in `Math` class.

Instance methods

- Fully resolved at run-time
- At compile time one call site could match several method definitions
- `void m(Object o){o.toString()}: o.toString()` matches all method definitions in all classes that match `String toString()`
- Naïve reachability analysis (RA) is sound, but grossly imprecise
- Problem: how to resolve possible types of an object?

CLASS HIERARCHY ANALYSIS (CHA)

Consider the type hierarchy

- Look at the declared type of a variable (target, receiver object)
- Restrict only to methods found in subtype of that type
- More precise than RA, and not that computationally expensive

Some types in the class hierarchy might never be instantiated/allocated

False positives

RAPID TYPE ANALYSIS (RTA)

A dispatch should be done on objects that are allocated (e.g., new ...)

Start with CHA CG

Construct a new CG

- Start with main
- Add only those edges for which types are allocated
- Methods that are reachable are added to the method processing queue
- Consider supertypes of the allocated types
- Methods can be re-evaluated several times since a different parameter type could be passed to them

RTA ALGORITHM

RTA = only method nodes

CHA = sound call graph

$W \leftarrow \text{main}$

while $Q \neq \emptyset$

- $m \leftarrow Q.\text{dequeue}()$
- $T \leftarrow \text{set of allocated types in } m$
- $T \leftarrow T \cup \text{allocated types in RTA callers of } M$ //main would have no callers
- *For each callsite* $c \in m$
 - *If* c is static then add edge to the resolved method
 - *else*
 - $m' \leftarrow \text{method } c \text{ called from } m \text{ in CHA}$
 - $m' \leftarrow m \cap \text{methods declared in } T \text{ or their supertypes}$
 - *for each* $m'' \in m'$: add edge between m and m'' and add m'' to Q

RTA ALGORITHM

More precise graph than CHA

- RTA does not add edges from CHA that are not feasible at runtime

Not sound

$T \leftarrow \text{set of allocated types in } m$

$T \leftarrow T \cup \text{allocated types in RTA callers of } M$

Unsoundness, only parents relations

```
public static void main(String[] args){
    Object o = foo();
    bar(o);
}

public static Object foo(){
    return new A();
}

public static void bar(Object o){
    o.toString()
}
```

```
public static Object v;

public static void main(String[] args){
    foo();
    bar();
}

public static void foo(){
    Object o = new A();
    v = o;
}

public static void bar(){
    v.toString()
}
```

Since neither *main* nor *bar* allocate any objects
 $T = \emptyset$ when invoking *bar* from *main*

FTA IMPROVEMENT OVER RTA

Field Type Analysis (FTA)

- Any method that reads from a field can inherit field-compatible allocated type of any method that could write to that field

```
public static Object v;  
  
public static void main(String[] args){  
    foo();  
    bar();  
}  
  
public static void foo(){  
    Object o = new A();  
    v = o;  
}  
  
public static void bar(){  
    v.toString()  
}
```

bar inherits A type from foo

MTA IMPROVEMENT OVER RTA

Method Type Analysis (return values and parameters)

- Types allocated in a method and then passed to a method through a parameter should be compatible with the called method's parameter types (improves precision)
- Return type should be added to the set of allocated types

```
public static void main(String[] args){
    Object o = foo();
    bar(o);
}

public static Object foo(){
    return new A();
}

public static void bar(Object o){
    o.toString()
}
```

main inherits A from *foo* and *bar* inherits it from *main*

XTA = MTA + FTA

XTA with ETA – handling exceptions

VTA

What happens to that code?

```
Object o = new A();  
o = new B();  
o.toString();
```

Imprecise since it connect main with toString of A and B

More precise Variable Type Analysis (VTA)

Requires points-to analysis

- Tracks the allocation site that each variable in the program could point-to
- Perfectly precise points-to analysis is simple not possible (halting problem reduces to it)
- A context-insensitive points-to analysis called 0-CFA

POINTS-TO ANALYSIS

Originally created for C

- Steensgaard's algorithm
- Andersen's algorithm (Thesis 1994) more precise than Steensgaard's
- Flow and context sensitivity

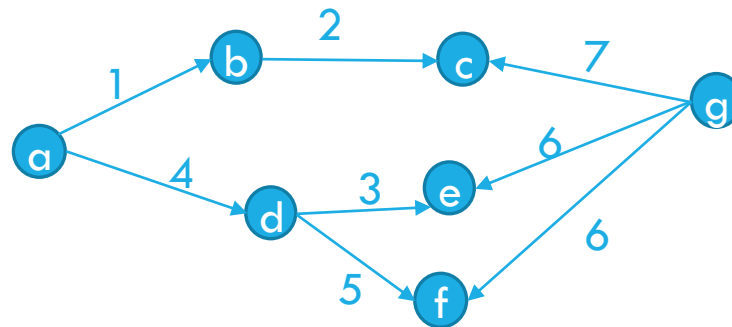
Java's extension

- Andresen
- Fields
- Objects
- Dynamic dispatches

ANDERSEN'S FOR C

```
int **a;  
int *b, *d, *g;  
int c, e, f;
```

1. `a = &b;`
2. `b = &c;`
3. `d = &e;`
4. `a = &d;`
5. `d = &f;`
6. `g = d;`
7. `g = *a;`



Find all pointer assignment in program

Form set of points-to graph nodes

Pointer variable/fields and variables on the heap

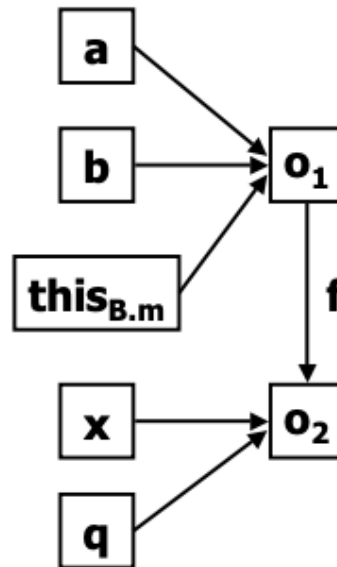
- For each statement (any order)
 - `p=q` create more edges: copy outgoing edges of `q` to `p`
 - if new edges are added to `q` then they also should be copied to `p`
- Parameters are treated like assignment statements

EXAMPLE POINTS-TO IN OO

```
class A { void m(X p) {..} }  
class B extends A {  
    X f;  
    void m(X q) { this.f=q;  
}
```

```
B b = new B();  
X x = new X();  
A a = b;  
a.m(x);
```

*A.m() not analyzed because
it's unreachable.*



Solution through a constraints on relations between points-to set of each variable.

Use fixed-point computation to solve it.

$O(n^3)$ worst case complexity

ADDING SENSITIVITIES

Context sensitivity

Field sensitivity

- Add object reference fields to points-to graph
- If class A has field g,f then $p = \text{new } A()$ adds $pf.$ and $p.g$ to the points-to graph
- Define and solve a system of annotated set-inclusion constraints
- Handles dynamic dispatch of run-time method lookup
- Models the fields of objects.

Client analysis:

- Object r/w information
- Synchronization removal
- Stack allocation

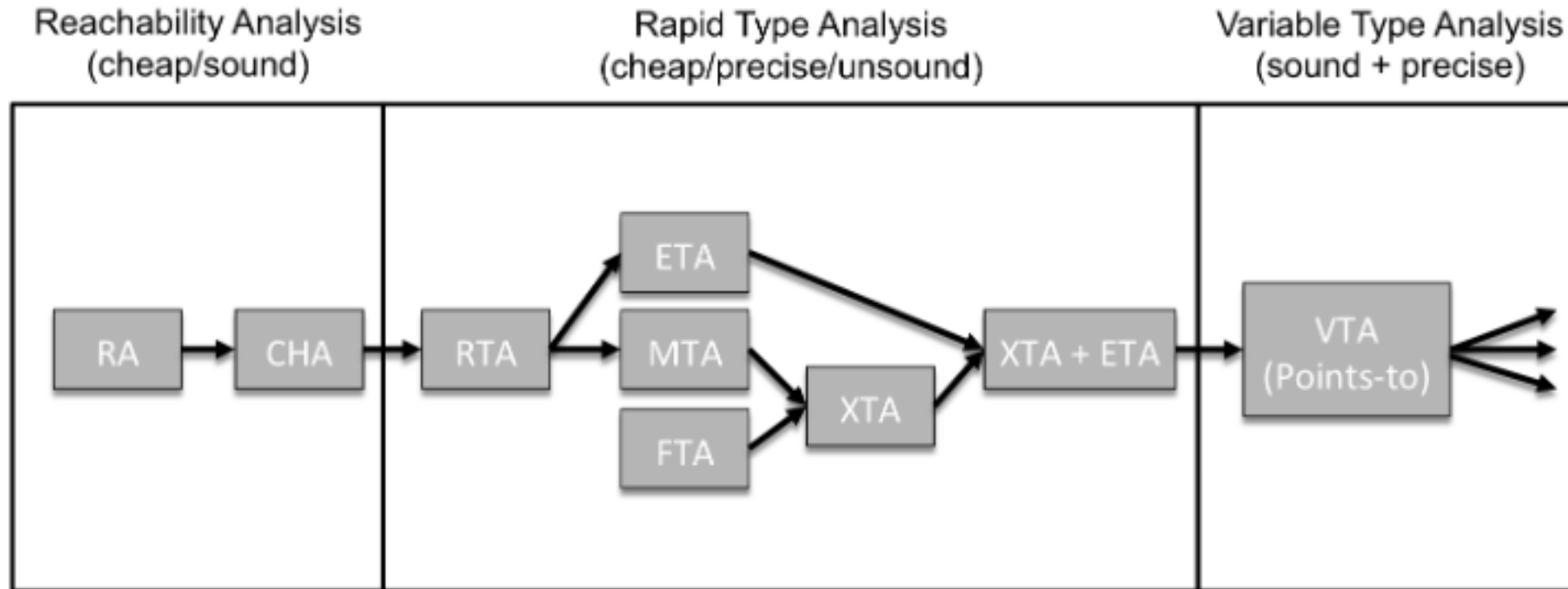
Object sensitivity

- Instance methods and constructors distinguished for different contexts
- Receiver objects used to distinguish calling contexts

Client analysis:

- Test coverage
- Program slicing
- Program comprehension

DIAGRAM OF CALL GRAPHS



source: <https://web.cs.ucla.edu/~palsberg/paper/oopsla00.pdf> and
<https://ben-holland.com/call-graph-construction-algorithms-explained>